Internal Dependencies

References

- Analyze java package metrics in a graph database
- Calculate metrics
- Neo4j Python Driver

1 - Modules

List the modules this notebook is based on. Different sorting variations help finding modules by their features and support larger code bases where the list of all modules gets very long.

Only the top 30 entries are shown. The whole table can be found in the following CSV report:
`List_all_Typescript_modules`

```
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotifica
tion.Statement.UnknownPropertyKeyWarning} {category: UNRECOGNIZED} {title: The provid
ed property key is not in the database} {description: One of the property names in yo
ur query is not available in the database, make sure you didn't misspell it or that t
he label is available when you run this statement in your application (the missing pr
operty name is: numberOfGitCommits)} {position: line: 9, column: 31, offset: 594} for
query: '// List all existing internal Typescript modules. Requires "Set_localRootPath
_for_modules.cypher", "Set_number_of...commits.cypher".\n \n MATCH (internalModule:T
S:Module)-[:EXPORTS]->(internalElement:TS)\n  WITH internalModule.name
AS moduleName\n      ,internalModule.rootProjectName                AS rootProjectNa
me\n      ,internalModule.localModuleDir                 AS localModuleDir\n      ,i
nternalModule.incomingDependencies            AS incomingDependencies\n      ,interna
lModule.outgoingDependencies            AS outgoingDependencies\n      ,coalesce(inte
rnalModule.numberOfGitCommits, 0) AS numberOfGitCommits\n      ,COUNT(DISTINCT intern
alElement.globalFqn)      AS numberOfElements\n RETURN rootProjectName, moduleName, n
umberOfElements, numberOfGitCommits, incomingDependencies, outgoingDependencies'
```

Table 1a - Top 30 modules with the highest element count

	rootProjectName	moduleName	numberOfElements	numberOfGitCommits	incomingDependencies	outgoingDependencies
0	react-router-6.30.2	react-router-dom	63	0	7	156
1	react-router-6.30.2	react-router-native	17	0	0	24
2	react-router-6.30.2	react-router	7	0	0	17
3	react-router-6.30.2	server	6	0	0	39

Table 1b - Top 30 modules with the highest number of incoming dependencies

The following table lists the top 30 internal modules that are used the most by other modules (highest count of incoming dependencies, highest in-degree).

	rootProjectName	moduleName	numberOfElements	numberOfGitCommits	incomingDependencies	outgoingDependencies
0	react-router-6.30.2	react-router-dom	63	0	7	156
1	react-router-6.30.2	react-router	7	0	0	17
2	react-router-6.30.2	react-router-native	17	0	0	24
3	react-router-6.30.2	server	6	0	0	39

Table 1c - Top 30 modules with the highest number of outgoing dependencies

The following table lists the top 30 internal modules that are depending on the highest number of other modules (highest count of outgoing dependencies, highest out-degree).

	rootProjectName	moduleName	numberOfElements	numberOfGitCommits	incomingDependencies	outgoingDependencies
0	react-router-6.30.2	react-router-dom	63	0	7	156
1	react-router-6.30.2	server	6	0	0	39
2	react-router-6.30.2	react-router-native	17	0	0	24
3	react-router-6.30.2	react-router	7	0	0	17

Table 1d - Top 30 modules with the lowest element count

	rootProjectName	moduleName	numberOfElements	numberOfGitCommits	incomingDependencies	outgoingDependencies
0	react-router-6.30.2	server	6	0	0	39
1	react-router-6.30.2	react-router	7	0	0	17
2	react-router-6.30.2	react-router-native	17	0	0	24
3	react-router-6.30.2	react-router-dom	63	0	7	156

Table 1e - Top 30 modules with the lowest number of incoming dependencies

The following table lists the top 30 internal modules that are used the least by other modules (lowest count of incoming dependencies, lowest in-degree).

	rootProjectName	moduleName	numberOfElements	numberOfGitCommits	incomingDependencies	outgoingDependencies
0	react-router-6.30.2	react-router	7	0	0	17
1	react-router-6.30.2	react-router-native	17	0	0	24
2	react-router-6.30.2	server	6	0	0	39
3	react-router-6.30.2	react-router-dom	63	0	7	156

Table 1f - Top 30 modules with the lowest number of outgoing dependencies

The following table lists the top 30 internal modules that are depending on the lowest number of other modules (lowest count of outgoing dependencies, lowest out-degree).

	rootProjectName	moduleName	numberOfElements	numberOfGitCommits	incomingDependencies	outgoingDependencies
0	react-router-6.30.2	react-router	7	0	0	17
1	react-router-6.30.2	react-router-native	17	0	0	24
2	react-router-6.30.2	server	6	0	0	39
3	react-router-6.30.2	react-router-dom	63	0	7	156

2 - Cyclic Dependencies

Cyclic dependencies occur when one module uses an elements of another module and vice versa. These dependencies can lead to problems when one of these modules needs to be changed.

Table 2a - Cyclic Dependencies Overview

Show the top 40 cyclic dependencies sorted by the most promising to resolve first. This is done by calculating the number of forward dependencies (first cycle participant to second cycle participant) in relation to backward dependencies (second cycle participant back to first cycle participant). The higher this rate (approaching 1), the easier it should be to resolve the cycle by focussing on the few backward dependencies.

Only the top 40 entries are shown. The whole table can be found in the following CSV report:
`Cyclic_Dependencies_for_Typescript`

Columns:

- *projectFileName* identifies the project of the first participant of the cycle
- *modulePathName* identifies the module of the first participant of the cycle
- *dependentProjectFileName* identifies the project of the second participant of the cycle
- *dependentModulePathName* identifies the module of the second participant of the cycle
- *forwardToBackwardBalance* is between 0 and 1. High for many forward and few backward dependencies.

- *numberForward* contains the number of dependencies from the first participant of the cycle to the second one
- *numberBackward* contains the number of dependencies from the second participant of the cycle back to the first one
- *someForwardDependencies* lists some forward dependencies in the text format "type1 -> type2"
- *backwardDependencies* lists the backward dependencies in the format "type1 <- type2" that are recommended to get resolved

projectFileName	moduleName	dependentProjectFileName	dependentModulePathName	forwardToBackwardBalance	numberForward	numbe

Table 2b - Cyclic Dependencies Break Down

Lists modules with cyclic dependencies with every dependency in a separate row sorted by the most promising dependency first.

Only the top 40 entries are shown. The whole table can be found in the following CSV report:
`Cyclic_Dependencies_Breakdown_for_Typescript`

Columns in addition to Table 2a:

- *dependency* shows the cycle dependency in the text format "type1 -> type2" (forward) or "type2<-type1" (backward)

projectFileName	moduleName	dependentProjectFileName	dependentModulePathName	dependency	forwardToBackwardBalance	numberFor

Table 2c - Cyclic Dependencies Break Down - Backward Dependencies Only

Lists modules with cyclic dependencies with every dependency in a separate row sorted by the most promising dependency first. This table only contains the backward dependencies from the second participant of the cycle back to the first one that are the most promising to resolve.

Only the top 40 entries are shown. The whole table can be found in the following CSV report:
`Cyclic_Dependencies_Breakdown_BackwardOnly_for_Typescript`

projectFileName	moduleName	dependentProjectFileName	dependentModulePathName	dependency	forwardToBackwardBalance	numberFor

3 - Module Usage

Table 3a - Elements that are used by multiple modules

This table shows the top 40 modules that are used by the highest number of different modules. The whole table can be found in the CSV report `WidelyUsedTypescriptElements`.

```
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotifica
tion.Statement.UnknownLabelWarning} {category: UNRECOGNIZED} {title: The provided lab
el is not in the database.} {description: One of the labels in your query is not avai
lable in the database, make sure you didn't misspell it or that the label is availabl
e when you run this statement in your application (the missing label name is: TestRel
ated)} {position: line: 3, column: 33, offset: 91} for query: "// List elements that
are used by many different modules\n \n MATCH (sourceModule:TS&Module&!TestRelated)-
[:EXPORTS]->(sourceElement:TS)\n MATCH (sourceElement)-[:DEPENDS_ON]->(dependentEleme
nt:TS&!Module&!ExternalModule)\n MATCH (dependentModule:TS&Module&!TestRelated)-[:EXP
ORTS]->(dependentElement)\n WHERE sourceModule <> dependentModule\n  WITH dependentEl
ement\n      ,labels(dependentElement)               AS dependentElementLabels\n
,COUNT(DISTINCT sourceModule.globalFqn) AS numberOfUsingModules\n UNWIND dependentEle
mentLabels AS dependentElementLabel\n   WITH *\n  WHERE NOT dependentElementLabel =
'TS' // Filter out generic TS label\n RETURN dependentElement.globalFqn            AS
fullQualifiedDependentElementName\n       ,dependentElement.moduleName           AS d
ependentElementModuleName\n       ,dependentElement.name                 AS dependent
ElementName\n       ,dependentElementLabel                 AS dependentElementLabels
\n       ,numberOfUsingModules\n  ORDER BY numberOfUsingModules DESC, dependentElemen
tName ASC\nLIMIT 40"
```

```
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotifica
tion.Statement.UnknownLabelWarning} {category: UNRECOGNIZED} {title: The provided lab
el is not in the database.} {description: One of the labels in your query is not avai
lable in the database, make sure you didn't misspell it or that the label is availabl
e when you run this statement in your application (the missing label name is: TestRel
ated)} {position: line: 5, column: 36, offset: 254} for query: "// List elements that
are used by many different modules\n \n MATCH (sourceModule:TS&Module&!TestRelated)-
[:EXPORTS]->(sourceElement:TS)\n MATCH (sourceElement)-[:DEPENDS_ON]->(dependentEleme
nt:TS&!Module&!ExternalModule)\n MATCH (dependentModule:TS&Module&!TestRelated)-[:EXP
ORTS]->(dependentElement)\n WHERE sourceModule <> dependentModule\n  WITH dependentEl
ement\n      ,labels(dependentElement)               AS dependentElementLabels\n
,COUNT(DISTINCT sourceModule.globalFqn) AS numberOfUsingModules\n UNWIND dependentEle
mentLabels AS dependentElementLabel\n   WITH *\n  WHERE NOT dependentElementLabel =
'TS' // Filter out generic TS label\n RETURN dependentElement.globalFqn            AS
fullQualifiedDependentElementName\n       ,dependentElement.moduleName           AS d
ependentElementModuleName\n       ,dependentElement.name                 AS dependent
ElementName\n       ,dependentElementLabel                 AS dependentElementLabels
\n       ,numberOfUsingModules\n  ORDER BY numberOfUsingModules DESC, dependentElemen
tName ASC\nLIMIT 40"
```

	fullQualifiedDependentElementName	dependentElementModuleName	dependentElementName	dependentElementLabels	numberOfUsingMod
0	"@remix-run/router".RouteObject	router	RouteObject	ExternalDeclaration	
1	"@remix-run/router".Router	router	Router	ExternalDeclaration	

Table 3b - Elements that are used by multiple modules

This table shows the top 30 modules that only use a few (compared to all existing) elements of another module. The whole table can be found in the CSV report `ModuleElementsUsageTypescript`.

```
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotifica
tion.Statement.UnknownLabelWarning} {category: UNRECOGNIZED} {title: The provided lab
el is not in the database.} {description: One of the labels in your query is not avai
lable in the database, make sure you didn't misspell it or that the label is availabl
e when you run this statement in your application (the missing label name is: TestRel
ated)} {position: line: 3, column: 33, offset: 122} for query: '// How many elements
compared to all existing are used by dependent Typescript modules?\n \n MATCH (source
Module:TS&Module&!TestRelated)-[:EXPORTS]-(sourceElement:TS)\n MATCH (sourceElement)-
[:DEPENDS_ON]->(dependentElement:TS&!Module&!ExternalModule)\n MATCH (dependentModul
e:TS&Module&!TestRelated)-[:EXPORTS]->(dependentElement)\n WHERE sourceModule <> depe
ndentModule\n MATCH (dependentModule)-[:EXPORTS]->(dependentModuleElement:TS)\n   WIT
H sourceModule.name                                    AS sourceModuleName\n       ,dep
endentModule.name                                   AS dependentModuleName\n       ,COUNT
(DISTINCT dependentElement.globalFqn)         AS dependentElementsCount\n       ,COUN
T(DISTINCT dependentModuleElement.globalFqn)   AS dependentModuleElementsCount\n
,collect(DISTINCT dependentElement.globalFqn)[0..4] AS dependentElementFullNameExampl
es\n       ,collect(DISTINCT dependentElement.name)[0..4]      AS dependentElementNam
eExamples\n RETURN sourceModuleName\n       ,dependentModuleName\n       ,dependentEl
ementsCount\n       ,dependentModuleElementsCount\n       ,toFloat(dependentElementsC
ount) / (dependentModuleElementsCount + 1E-38) AS elementUsagePercentage\n       ,dep
endentElementFullNameExamples\n       ,dependentElementNameExamples\n ORDER BY elemen
tUsagePercentage ASC\nLIMIT 30'
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotifica
tion.Statement.UnknownLabelWarning} {category: UNRECOGNIZED} {title: The provided lab
el is not in the database.} {description: One of the labels in your query is not avai
lable in the database, make sure you didn't misspell it or that the label is availabl
e when you run this statement in your application (the missing label name is: TestRel
ated)} {position: line: 5, column: 36, offset: 284} for query: '// How many elements
compared to all existing are used by dependent Typescript modules?\n \n MATCH (source
Module:TS&Module&!TestRelated)-[:EXPORTS]-(sourceElement:TS)\n MATCH (sourceElement)-
[:DEPENDS_ON]->(dependentElement:TS&!Module&!ExternalModule)\n MATCH (dependentModul
e:TS&Module&!TestRelated)-[:EXPORTS]->(dependentElement)\n WHERE sourceModule <> depe
ndentModule\n MATCH (dependentModule)-[:EXPORTS]->(dependentModuleElement:TS)\n   WIT
H sourceModule.name                                    AS sourceModuleName\n       ,dep
endentModule.name                                   AS dependentModuleName\n       ,COUNT
(DISTINCT dependentElement.globalFqn)         AS dependentElementsCount\n       ,COUN
T(DISTINCT dependentModuleElement.globalFqn)   AS dependentModuleElementsCount\n
,collect(DISTINCT dependentElement.globalFqn)[0..4] AS dependentElementFullNameExampl
es\n       ,collect(DISTINCT dependentElement.name)[0..4]      AS dependentElementNam
eExamples\n RETURN sourceModuleName\n       ,dependentModuleName\n       ,dependentEl
ementsCount\n       ,dependentModuleElementsCount\n       ,toFloat(dependentElementsC
ount) / (dependentModuleElementsCount + 1E-38) AS elementUsagePercentage\n       ,dep
endentElementFullNameExamples\n       ,dependentElementNameExamples\n ORDER BY elemen
tUsagePercentage ASC\nLIMIT 30'
```

	sourceModuleName	dependentModuleName	dependentElementsCount	dependentModuleElementsCount	elementUsagePercentage	depende
0	server	react-router-dom	2	63	0.031746	["@re

Table 3c - Distance distribution between dependent files

This table shows the file directory distance distribution between dependent files. Intuitively, the distance is given by the fewest number of change directory commands needed to navigate between a file and a dependency it uses. Those are aggregate to see how many dependent files are in the same directory, how many are just one change directory command apart, and so on.

	dependency.fileDistanceAsFewestChangeDirectoryCommands	numberOfDependencies	numberOfDependencyUsers	numberOfDependencyProv
0	0	1	1	